Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments

Daniel Torres, Pranav A. Bhounsule

Department of Mechanical and Industrial Engineering, University of Illinois at Chicago, 842 W Taylor St, Chicago, IL 60607, USA. (e-mail: dtorre38@uic.edu; pranav@uic.edu)

Abstract: Most planners focus on obstacle avoidance, but legged robots can step over obstacles, reducing travel distance and time. We address this gap by integrating motion control and planning within a sampling-based planner. We develop a constrained inverse-dynamics controller with task-space feedback that tracks center-of-mass velocities via a closed-form solution, thereby avoiding iterative optimization. This enables the use of reduced-order planning models by ensuring dynamic consistency at the full-body level. For planning, we introduce BiKE-Legs and DRiVE-Legs, which augment classical bicycle and differential-drive kinematics with leg-based stepping capabilities. These models are embedded within the Dynamic Window Approach (DWA), with cost functions that balance goal progress, obstacle negotiation, and speed to enable selection between avoidance and stepping. Results across scenarios with small and large obstacles using reduced kinematic models, along with limited validation on a simulated Digit humanoid, demonstrate the effectiveness of the proposed approach.

Keywords: Humanoid, Inverse Dynamics, Dynamic Window Approach, Reduced models.

1. INTRODUCTION

Legged robots offer the unique ability to place discrete footholds (see Fig. 1), enabling them to go around large obstacles (tall blue cylinders) but save time by stepping over small obstacles (short blue obstacles) to move from the start (green cylinder) to a goal (red cylinder). However, relatively little work has explored stepping over obstacles, as this requires explicit planning of foot placements while accounting for the robot's dynamics. To address this problem, we use a hierarchical approach. At the control level, a computationally efficient constrained inverse dynamics method tracks center-of-mass velocity. At the planning level, 2D kinematic models with leg extensions are integrated into a sampling-based Dynamic Window Approach (DWA). This configuration enables both stepping-over and stepping-around behaviors, allowing the robot to move effectively through cluttered environments.

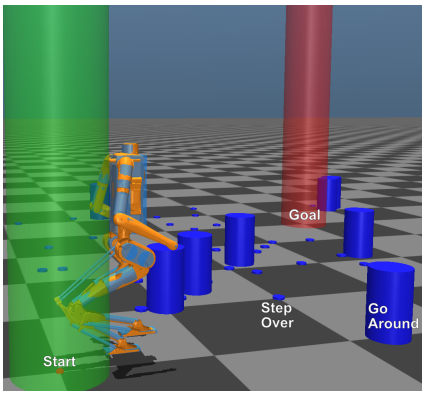


Fig. 1. **Overview:** The BiKE-Legs model (bicycle kinematics enhanced with legs) is used along with the Dynamic Window Approach, a sampling-based approach for motion planning on the humanoid Digit in the MuJoCo simulator.

Motion planning for legged robots requires an underlying model to represent the robot's motion. The simplest models are quasi-static models, which allow for fast computation Hauser et al. (2008). While computationally efficient, such models often fail to adequately capture the system's dynamics. At the other extreme, full-dynamics models account for nonlinear and hybrid dynamics, but their use in real-time planning is limited by high computational demands Kuindersma et al. (2016). Between these two extremes, reduced-order models such as the Linear Inverted Pendulum Model (LIPM) for walking Peng et al. (2024) and the Spring-Loaded Inverted Pendulum (SLIP) model of running Zamani and Bhounsule (2020) provide a useful compromise by retaining essential dynamic behaviors while simplifying computations.

Global motion planning for legged robots has been approached through a variety of search- and sampling-

⋆ This material is based upon work supported by the National Science Foundation under Grant No. 2128568 to P. A. Bhounsule and by the National Science Foundation Graduate Research Fellowship Program under Grant No. 2236870 to D. Torres. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

based frameworks that account for terrain feasibility, dynamic feasibility, and locomotion efficiency. Early work demonstrated that footstep primitives derived from full-body simulations could be integrated into an A^* framework, where costs for terrain difficulty, turning, and lateral/longitudinal stepping were optimized to yield energy-efficient step sequences for humanoids Huang et al. (2013). More recent efforts have leveraged reduced-order models to generate feasible motion primitives that can be embedded in sampling-based planners, such as RRT-Connect, and that respect both kinematic and dynamic constraints, enabling fast global planning of footstep locations Norby and Johnson (2020). Beyond feasibility and efficiency, safety considerations have also been incorporated, as in the safety-aware informative planners that integrate control barrier functions into the tree expansion process to balance safety with information gain Teng et al. (2021). Hybrid approaches such as the Long Short-Term Planning (LSTP) framework Jelavic et al. (2023), which first samples contact sequences and schedules globally, and then refines the solution locally with short-horizon trajectory optimization, offer the best of both worlds.

In the absence of a global map or in changing environments, legged robots should be able to utilize local motion planning for real-time control. Model Predictive Control (MPC) has become a widely adopted approach, where reduced-order models such as the Linear Inverted Pendulum are employed to generate short-horizon trajectories with automatic footstep placement, enabling stable walking on uneven terrain Herdt et al. (2010). Trajectory optimization methods have also been applied locally, with formulations that continuously refine body and footstep trajectories subject to contact and dynamic constraints Kuindersma et al. (2016). Reinforcement learning has provided an alternative, with end-to-end policies trained in simulation to react to local terrain features, producing robust and agile legged behaviors Peng et al. (2017); Hwangbo et al. (2019).

The Dynamic Window Approach (DWA) provides a computationally efficient method for local motion planning by sampling admissible velocity commands and selecting the one that minimizes a cost that weighs goal progress, obstacle avoidance, and speed Fox et al. (1997). DWA and its variants have been widely used for car-like and differential-drive navigation Brock and Khatib (1999); Ogren and Leonard (2005), and more recently applied directly to multi-legged robot navigation Zhang et al. (2024), though without an explicit stepping-over capability. In this work, we extend the DWA to legged locomotion by introducing leg extensions to the Bicycle Kinematics model (BiKE-Legs) and differential drive model (DRiVE-Legs) and by designing cost functions that explicitly permit stepping around and over obstacles. Unlike MPC and trajectory-optimization approaches Herdt et al. (2010); Kuindersma et al. (2016), which do not explicitly choose between avoiding and stepping over an obstacle, and unlike global footstep-sampling planners Huang et al. (2013); Norby and Johnson (2020), which require a known map and offline planning, our approach embeds the avoid-or-step decision into a real-time local planner's cost function with no global map and no per-step optimization. Our contributions are as follows: (1) a computationally efficient constrained inverse-dynamics controller for high-fidelity tracking of center-of-mass velocities without online optimization; (2) two leg-enhanced kinematic planning models based on bicycle and differential-drive kinematics; and (3) extensions of DWA that enable obstacle negotiation via stepping, a capability not previously realized in DWA-style local planners. We validate our approach using Agility Robotics' Digit, simulated in MuJoCo Todorov et al. (2012).

2. METHODS

2.1 Motion Control: Constrained Inverse Dynamics

Digit Model: The reduced lower-body model shown in Fig. 2 has 22 generalized coordinates: six floating-base coordinates $[q_x, q_y, q_z, q_\phi, q_\theta, q_\psi]$ and eight coordinates $[q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8]$ in each leg. Each leg has six actuated DoFs and two passive coordinates, q_5, q_6, associated with the closed-chain leg mechanism. The upper body and torso are lumped as a single rigid body with a combined mass and inertia. The robot's total weight is 48 kg, with about 15 kg in the torso. The robot torso houses the inertial measurement unit (IMU), and each joint has an encoder that provides the position and velocity. The controller runs at 1 kHz.

State Machine As shown in Fig. 2, a state machine takes inputs of fore-aft speed $\dot{x}_{\text{ref}}$, lateral speed $\dot{y}_{\text{ref}}$, and yaw rate $\dot{\psi}_{\text{ref}}$ from the motion planner (once per step) and the robot state $\mathbf{q}, \dot{\mathbf{q}}$ (at 1 kHz). The state machine keeps track of phases and transitions (s =single stance/double stance/swing leg) and sets up start and end positions for the swing foot and the torso $\ell_{\text{ref}}, \dot{\ell}_{\text{ref}}$. We assume that the single stance phase is fixed at $T = 0.4$ s and the double stance phase depends on the estimated vertical force in the rear leg computed from the joint control discussed in this section.

Cartesian and Joint Reference Trajectory: The Cartesian trajectory generator computes the swing foot and torso positions using a fifth-order polynomial with the given initial and final positions from the state machine, and with the initial/final velocities and accelerations set to zero. The joint trajectory generator uses a semi-analytical approach to compute the joint reference angle from the end-effector reference position. The simplification for inverse kinematics is as follows. The loop joint on the leg is approximated as a parallel linkage. Consequently, we solve for the hip pitch q_3 and the knee angle q_4 based on the desired leg length and fore-aft leg position using a root-finding method. Thereafter, the remaining four angles q_1, q_2, q_7, q_8 can be obtained analytically from the desired end-effector angles and the lateral position of the foot. Once the joint reference angles are solved, we compute the joint reference velocities using the Jacobian $\mathbf{J}$ from foot to torso as follows: $\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1}\dot{\ell}$.

Joint Control: The joint control is based on constrained inverse dynamics with task-space feedback, which computes joint torques and constraint forces based on reference joint angles/rates and center-of-mass tracking errors. The dynamics of the Digit are given by

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \mathbf{S}^\top \boldsymbol{\tau} + \mathbf{J}^\top \mathbf{F}, \qquad (1)$$

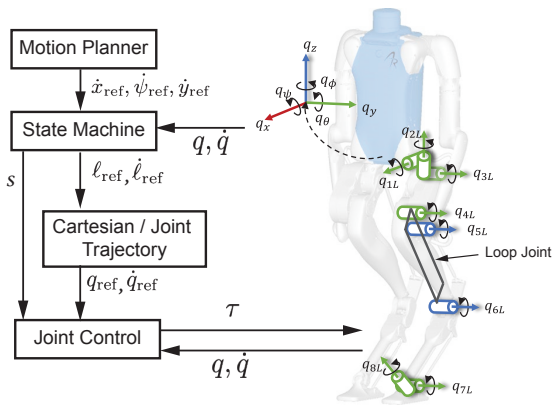


Fig. 2. **Digit model:** The reduced lower-body model has 22 generalized coordinates: six floating-base coordinates and eight coordinates in each leg. Each leg has six actuated DoFs and two passive coordinates associated with the closed-chain leg mechanism. Figure adapted from Castillo et al. (2021).

where $\mathbf{q} \in \mathbb{R}^{22}$ is the generalized coordinates, the inertia matrix is $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{22\times 22}$, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} \in \mathbb{R}^{22}$ represents Coriolis and centrifugal terms, and $\mathbf{G}(\mathbf{q}) \in \mathbb{R}^{22}$ is the gravity vector. The actuation selection matrix is $\mathbf{S} \in \mathbb{R}^{12\times 22}$, such that $\boldsymbol{\tau} \in \mathbb{R}^{12}$ are the joint torques applied at the actuated joints. The contact Jacobian is $\mathbf{J}(\mathbf{q})$, written as $\mathbf{J}$ for brevity elsewhere in this paper, and $\mathbf{F} \in \mathbb{R}^{10}$ denotes the vector of external forces, including 6 contact forces at the feet and 4 constraint forces due to the loop joint on the legs.

The generalized coordinates are partitioned into floating base f and leg ℓ components as $\mathbf{q} = [\mathbf{q}_f, \mathbf{q}_\ell]^\top$, $\mathbf{q}_f \in \mathbb{R}^6$, $\mathbf{q}_\ell \in \mathbb{R}^{12}$. Here, $\mathbf{q}_\ell$ contains only the 6 actuated degrees of freedom per leg (12 total); the passive coordinates are excluded from this partition and are instead enforced through the loop-joint constraint forces in $\mathbf{F}$, consistent with the full 22-dimensional $\mathbf{q}$ used in (1). To control the floating-base motion, the dynamics in (1) are partitioned into floating base and leg components:

$$\begin{aligned} \mathbf{M}_f(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}_f(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}_f(\mathbf{q}) &= \mathbf{J}_f^\top \mathbf{F}, \\ \mathbf{M}_\ell(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}_\ell(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}_\ell(\mathbf{q}) &= \tau_g + \mathbf{J}_\ell^\top \mathbf{F}. \end{aligned} \tag{2}$$

The control objective is to regulate the floating-base motion in task space. Let $\mathbf{X}$ denote the floating-base pose, and define tracking errors $\Delta\mathbf{X} = \mathbf{X}_{\text{ref}} - \mathbf{X}$ and $\Delta\dot{\mathbf{X}} = \dot{\mathbf{X}}_{\text{ref}} - \dot{\mathbf{X}}$. A desired task-space acceleration is specified using a proportional-derivative control law:

$$\begin{aligned} \ddot{\mathbf{X}}_{\text{des}} &= -\mathbf{K}_*\Delta\mathbf{X} - \mathbf{C}_*\Delta\dot{\mathbf{X}} \\ &= \begin{bmatrix} -C_x(\dot{x} - \dot{x}_{\text{ref}}), -C_y(\dot{y} - \dot{y}_{\text{ref}}), \dots \\ -K_z(\bar{z} - z) - C_z\dot{z}, -K_\phi\phi - C_\phi\dot{\phi}, \dots \\ -K_\theta\theta - C_\theta\dot{\theta}, -C_\psi(\dot{\psi} - \dot{\psi}_{\text{ref}}) \end{bmatrix}^\top \end{aligned}$$

where $\mathbf{K}_*$ and $\mathbf{C}_*$ are user-chosen gains, $\bar{z}$ is the reference height of the torso from the ground. This is incorporated into the floating-base dynamics, resulting in the following task-space feedback:

$$\mathbf{S}^\top\tau_g + \mathbf{J}^\top\mathbf{F} = \begin{bmatrix} -\mathbf{K}_*\Delta\mathbf{X} - \mathbf{C}_*\Delta\dot{\mathbf{X}} + \mathbf{G}_f(\mathbf{q}_{\text{ref}}) \\ \mathbf{M}_\ell(\mathbf{q}_{\text{ref}})\ddot{\mathbf{q}}_{\text{ref}} + \mathbf{C}_\ell(\mathbf{q}_{\text{ref}}, \dot{\mathbf{q}}_{\text{ref}})\dot{\mathbf{q}}_{\text{ref}} + \mathbf{G}_\ell(\mathbf{q}_{\text{ref}}) \end{bmatrix}$$

where $\mathbf{q}_{\text{ref}}, \dot{\mathbf{q}}_{\text{ref}}, \ddot{\mathbf{q}}_{\text{ref}}$ denote reference joint trajectories. For simplicity, we set $\mathbf{C}_\ell(\mathbf{q}_{\text{ref}}, \dot{\mathbf{q}}_{\text{ref}})\dot{\mathbf{q}}_{\text{ref}} = \mathbf{0}$ and $\mathbf{M}_\ell(\mathbf{q})\ddot{\mathbf{q}}_{\text{ref}} = \mathbf{0}$ in our controller. The equations can then be simplified and solved for the unknowns $\lambda = [\tau_g, \mathbf{F}]$:

$$\mathbf{A}\lambda = \mathbf{b} \quad \Rightarrow \lambda = \mathbf{A}^+\mathbf{b} \tag{3}$$

where $\mathbf{b} = \left[-\mathbf{K}_*\Delta\mathbf{X} - \mathbf{C}_*\Delta\dot{\mathbf{X}} + \mathbf{G}_f(\mathbf{q}_{\text{ref}}), \mathbf{G}_\ell(\mathbf{q}_{\text{ref}})\right]^\top$, $\mathbf{A} = [\mathbf{S}^\top, \mathbf{J}^\top]$, and $\mathbf{A}^+$ is the pseudoinverse of $\mathbf{A}$. The forces $\mathbf{F}$ and consequently the Jacobian $\mathbf{J}$ in these equations are modified based on the phase as follows:

- **Double stance: F** consists of contact forces at both feet (3 forces per foot) and loop-closure constraint forces (2 per leg), yielding $\mathbf{F} \in \mathbb{R}^{10}$.
- **Single stance: F** consists of a 6D contact wrench (3 forces and 3 moments) at the stance foot, along with loop-closure constraint forces (2 per leg), again yielding $\mathbf{F} \in \mathbb{R}^{10}$.

Equation (3) is solved to obtain the torques τ_g, which provide gravity compensation, and the forces $\mathbf{F}$, which include constraint and reaction forces. Some of these forces may be used for transitions from double stance to single stance and for safety checks, such as detecting whether the robot is falling. Finally, to enable stable joint control, additional feedback torques are added at each joint as follows.

$$\tau = \tau_g + \mathbf{K}_q(\mathbf{q}_{\text{ref}} - \mathbf{q}) + \mathbf{C}_q(\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}}) \tag{4}$$

where $\mathbf{K}_q$ and $\mathbf{C}_q$ are user-defined position and derivative gains at the joint, respectively.

2.2 Motion Planning: Dynamic Window Approach

Review of Two Kinematic Models: The kinematic bicycle model and the differential-drive car model LaValle (2006) are given by the following equations.

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta,$$

$$\dot{\theta} = \begin{cases} \dfrac{v}{L}\tan\phi, & \text{(bicycle kinematics, controls } v, \phi) \\ \omega, & \text{(differential-drive car, controls } v, \omega) \end{cases}$$

Here, the center of mass has global position (x, y) and heading or forward direction with respect to the x-axis θ. Both models share the forward body-frame speed v as a control input. The bicycle model uses steering angle ϕ, whereas the differential-drive model uses the angular speed ω as the second control input. In the bicycle model, we define the wheelbase L as the distance between the front and rear axles.

Two Kinematic Models with Leg Enhancement: Figure 3 shows the modifications to these bicycle and differential drive models to obtain the BiKE-Legs model, **Bi**cycle **K**inematics **E**nhanced with **Legs** and DRiVE-Legs, the **D**ifferential d**RiV**e **E**nhanced with **Legs**, respectively. The center of mass is located at (x, y). The orientation of the x-axis of the body frame (dashed line) is at an angle θ, the heading angle, to the global x-axis (points to the right). The bipedal robot has made contact with the front foot at (x_f, y_f) with a step length of L. Since we are showing distance from the center of mass to the foot, this corresponds to half the step length, $0.5L$, as shown in the figure. For the BiKE-Legs model, the leg orientation is at an angle $\psi = \phi$ to the body heading angle, as shown. The DRiVE-Legs model instead uses the leg orientation angle $\psi = \omega T$, where T is the step time. Both models define the

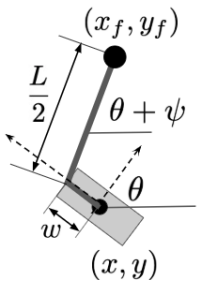


Fig. 3. **Two kinematic models for planning (top-down view):** The center of mass is (x, y) and foot position is (x_f, y_f). The body frame shown using dashed lines is at an angle θ to the global x-axis (not shown). w is the half body width, the lateral offset from the center of mass to the body edge. In the BiKE-Legs, the **Bi**cycle **K**inematics **E**nhanced with **Legs** $\psi = \phi$ and the DRiVE-Legs, the **D**ifferential d**RiV**e **E**nhanced with **Legs** $\psi = \omega T$ with step time T.

step velocity as $v = \frac{L}{T}$. The equations of the BiKE-Legs and DRiVE-Legs models are given by

$$\dot{x} = \frac{L}{T}\cos\theta,\ \dot{y} = \frac{L}{T}\sin\theta, \tag{5}$$

$$x_f = x - s\,w\,\sin\theta + L\cos(\theta + \psi), \tag{6}$$

$$y_f = y + s\,w\,\cos\theta + L\sin(\theta + \psi) \tag{7}$$

$$\dot{\theta} = \begin{cases} \frac{1}{T}\tan\phi,\ \psi = \phi & \text{BiKE-Legs} \\ \omega,\ \psi = \omega T & \text{DRiVE-Legs} \end{cases} \tag{8}$$

where $s = 1$ if the left leg makes contact with the ground and $s = -1$ if the right leg makes contact with the ground. Once the foot makes contact with the ground, it stays at that location for a duration equal to the step time T.

Dynamic Window Approach (DWA) The DWA minimizes a cost over a horizon $k = 1, 2, \ldots, N$ steps subject to bounds on the controls.

$$\textbf{BiKE-Legs:} \quad \min_{\{L_k, \phi_k\}_{k=1}^N} \ J = \sum_{k=1}^{N} C(L_k, \phi_k) \tag{9}$$

$$\text{s.t.} \quad L_{\min} \le L_k \le L_{\max}, \quad \phi_{\min} \le \phi_k \le \phi_{\max}, \tag{10}$$

$$\textbf{DRiVE-Legs:} \quad \min_{\{L_k, \omega_k\}_{k=1}^N} \ J = \sum_{k=1}^{N} C(L_k, \omega_k) \tag{11}$$

$$\text{s.t.} \quad L_{\min} \le L_k \le L_{\max}, \quad \omega_{\min} \le \omega_k \le \omega_{\max}, \tag{12}$$

Let the goal position be (x_g, y_g), and each circular obstacle projection is parameterized by a center (x_i, y_i) and radius r_i. If the Euclidean distance between two points (x_a, y_a) and (x_b, y_b) is given by $d(a, b) = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$ then

$$C = C_{\text{goal}} + \alpha C_{\text{speed}} + \beta C_{\text{body}} + \gamma C_{\text{foot}}, \tag{13}$$

$$C_{\text{goal}} = d(x, y;\, x_g, y_g), \qquad C_{\text{speed}} = -L, \tag{14}$$

$$C_{\text{body}} = \left| \min_i \left(0,\ d(x, y;\, x_i, y_i) - r_i - r_b^{(i)}\right) \right|, \tag{15}$$

$$C_{\text{foot}} = \left| \min_i \left(0,\ d(x_f, y_f;\, x_i, y_i) - r_i - r_b^{(i)}\right) \right|, \tag{16}$$

$$r_b^{(i)} = \begin{cases} r_{\text{buffer}}, & r_i > r_{\text{thresh}} \quad \text{(large obstacle)}, \\ 0, & r_i \le r_{\text{thresh}} \quad \text{(small obstacle)}. \end{cases} \tag{17}$$

The total cost C in Eqn. 13 is composed of four terms. C_{goal} (Eqn. 14) penalizes distance to the target position (x_g, y_g), encouraging progress toward the goal. C_{speed} (Eqn. 14) rewards larger step lengths L, promoting faster motion. C_{body} (Eqn. 15) penalizes collisions of the robot's body with large obstacles, where a buffer $r_b^{(i)}$ (Eqn. 17) is added to obstacle radii to maintain safe clearance. C_{foot} (Eqn. 16) penalizes collisions of the robot's feet with small obstacles, allowing the planner to account for stepping over them; no additional buffer is added in this case. The weighting factors α, β, γ balance the contributions of speed and collision avoidance relative to goal-reaching.

Mapping the Two Kinematic Models to Humanoid: In the humanoid model shown in Fig. 2, we have three control inputs from the motion planner. These are the fore-aft speed $\dot{x}_{\text{ref}}$, the lateral speed $\dot{y}_{\text{ref}}$, and the yaw rate $\dot{\psi}_{\text{ref}}$. These are used as follows.

BiKE-Legs constrains yaw motion $\dot{\psi}_{\text{ref}} = 0$ while the other two velocities are mapped to L, ϕ as follows.

$$L = T\sqrt{\dot{x}_{\text{ref}}^2 + \dot{y}_{\text{ref}}^2}, \quad \phi = \text{atan2}(\dot{y}_{\text{ref}}, \dot{x}_{\text{ref}}). \tag{18}$$

DRiVE-Legs constrains the lateral motion $\dot{y}_{\text{ref}} = 0$, while the other two velocities are mapped to L, ω as follows.

$$L = T\dot{x}_{\text{ref}}, \quad \omega = \dot{\psi}_{\text{ref}}. \tag{19}$$

3. RESULTS

Motion Planning Using Kinematic Models in Python: All terrains were generated in a 4×4 grid cluttered with N_{obs} user-specified circular obstacles. A fraction f_{obs} were large obstacles with radius $r_0 = 0.15$ m, and the remaining were small obstacles with $r_0 = 0.05$ m. It is assumed that the large obstacles should be avoided, as they are too tall or wide for the robot to step over, and that the small obstacles can be stepped over. Obstacle height is not modeled explicitly in this 2D framework; radius is used as a proxy for step-over feasibility, a simplification discussed further in Section 4.

For all simulations, we used a start location $x(0) = y(0) = -1.8$ m and orientation $\theta(0) = \frac{\pi}{4}$ rad, and goal location $x_g = y_g = 1.8$ m. The controller was updated at a fixed time step of $T = 0.4$ s, and the dynamics were integrated via Euler's method with $dt = 0.02$ s. Half the body width is $w = 0.125$ m, as shown in Fig. 3. The step length bounds are $L_{\min} = 0.1$ m and $L_{\max} = 0.5$ m, the orientation bounds are $\phi_{\min} = -\frac{\pi}{4}$ rad, $\phi_{\max} = \frac{\pi}{4}$ rad, and the angular velocity bounds are $\omega_{\min} = -2$

rad/s, $\omega_{\max} = 2$ rad/s. The algorithm terminates when the step count reaches the maximum $N_{\max} = 50$ or $\sqrt{(x(N) - x_g)^2 + (y(N) - y_g)^2} < 1.5 r_g$ where $r_g = 0.15$ m, whichever is earlier. We use the following parameters for DWA: $\alpha = 10$, $\beta = \gamma = 100$, $r_{\text{buffer}} = 0.05$ m, $r_{\text{thresh}} = 0.1$ m. Table 1 sweeps three parameters (N_{obs}, f_{obs}, N_{plan}), each averaged over 5 random scenes, on a 2.8 GHz Intel Core i7 MacBook in Python 3.9.

Table 1 reports average step count, compute time, average control parameters, and stretch factor (SF), the ratio of actual path length to the Euclidean distance to goal.

At baseline ($N_{\text{obs}} = 60$, $f_{\text{obs}} = 0.2$, $N_{\text{plan}} = 2$), BL and DL perform comparably across all metrics, with SF near 1.1 for both, indicating near-straight trajectories.

With $N_{\text{obs}} = 120$, both models take more steps and compute time, but BL degrades more sharply: 18.6 steps and $SF = 1.76\times$ vs. DL's 13.8 steps and $SF = 1.31\times$, suggesting BL is more sensitive to clutter under the same planning horizon.

With $f_{\text{obs}} = 0.4$, stretch factor increases moderately for both models but less severely than raising N_{obs}, suggesting obstacle count has more impact on path quality than size distribution.

Varying the planning horizon reveals a cost-quality trade-off. $N_{\text{plan}} = 4$ nearly doubles compute time without improving SF (matched baseline: $1.08\times/1.13\times$; $N_{\text{plan}} = 4$: $1.40\times/1.22\times$). The reactive controller ($N_{\text{plan}} = 1$) halves compute time but yields slightly higher step counts and SF.

Across all conditions, the average step length stays near its upper bound (0.5 m), meaning both planners use maximum stride when possible. The average heading (BL) and angular velocity (DL) stay near zero, consistent with the robot starting roughly oriented toward the goal. Under the current Python implementation, only the reactive controller ($N_{\text{plan}} = 1$) consistently meets the 1 Hz control loop rate. The DWA uses a greedy per-step rollout rather than a joint horizon search, so cost scales linearly with N_{plan}: 4 samples of L and 8 of ϕ or ω give 32 evaluations per step, so $N_{\text{plan}} = 4$ requires roughly twice the computation time of $N_{\text{plan}} = 2$, matching Table 1. A compiled implementation (C/C++) is expected to reduce per-evaluation cost substantially; Python-to-compiled speedups of one to two orders of magnitude are typical for sample-and-evaluate loops of this kind.

Motion Control and Planning on Digit Humanoid: All Digit simulations were performed using the C version of MuJoCo 2.0, while the planner ran in Python 3.9 via a client-server framework.

Figure 4 shows tracking of changing fore-aft, lateral, and yaw-rate velocity references (dashed, input mid-step) against step-averaged actual velocities (solid red). Yaw-rate tracking is high-fidelity; fore-aft and lateral show errors of about 0.1 m/s and 0.2 m/s, respectively. Higher gains would reduce these errors but increase instability risk.

Next, we verified the BiKE-Legs kinematic model in simulation. We had to reduce the fore-aft step length to 0.2 m while keeping the step time at 0.4 s, as larger step lengths led to instability. As observed in our earlier work, the lateral motion is less stable than the fore-aft motion in bipeds Hinojosa and Bhounsule (2023). This implies that if the robot takes a larger side step (large ϕ), it should decrease its speed to ensure stable walking. To circumvent this, we added a maneuver cost $C_{\text{maneuver}} = C_m L |\phi|^p$, with $p = 1, 2, \ldots$, where C_m is a constant and this term is added to Eqn. 13. A higher p indicates a more severe coupling. Figure 5 shows results for Digit in simulation for $N_{\text{obs}} = 60$, $f_{\text{obs}} = 0.2$, $N_{\text{plan}} = 3$.

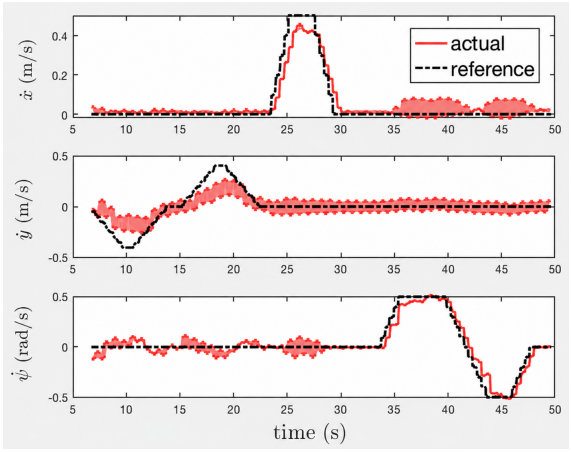


Fig. 4. **Motion Control:** Tracking velocity of the center of mass using constrained inverse dynamics with task-space feedback control.

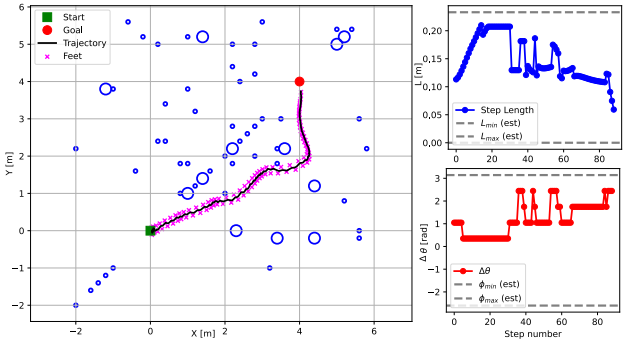


Fig. 5. Results for $N_{\text{obs}} = 60$, $f_{\text{obs}} = 0.2$, $N_{\text{plan}} = 3$ with C_{maneuver} for the BiKE-Legs model in MuJoCo. Small circles mark obstacles the planner may step over; large circles mark obstacles it must avoid. The trajectory and foot markers pass directly over the small obstacles near $(2, 1)$ and $(2, 2)$, demonstrating stepping-over, while visibly routing around every large obstacle along the path.

4. CONCLUSION AND FUTURE WORK

This paper shows that a constrained inverse dynamics controller with task-space feedback can track center-of-mass reference motions well enough to support 2D kinematic planning models, BiKE-Legs and DRiVE-Legs, for stepping over and around obstacles within the DWA framework.

The current 2D model does not account for obstacle height, vertical foot clearance, or kinematic reachability; using

Type	Variable	BL	DL	BL	DL	BL	DL	BL	DL	BL	DL
Control	Total Obstacles (N_{obs})	60	60	120	120	60	60	60	60	60	60
	Fraction Big Obstacles (f_{obs})	0.20	0.20	0.20	0.20	0.40	0.40	0.20	0.20	0.20	0.20
	Steps for Planning (N_{plan})	2	2	2	2	2	2	4	4	1	1
Metrics	Avg. Steps	11.20	12.00	18.60	13.80	12.00	12.40	14.40	12.60	14.40	12.20
	Avg. Compute Time (s)	0.49	0.46	0.92	0.92	0.48	0.45	0.88	0.91	0.23	0.23
	Max. Compute Time (s)	0.62	0.48	1.19	1.05	0.75	0.54	1.00	1.07	0.37	0.25
	Avg. Step Length (m)	0.50	0.49	0.49	0.48	0.50	0.49	0.50	0.49	0.45	0.47
	Avg. Heading (rad)	0.04	–	0.02	–	0.06	–	-0.02	–	-0.01	–
	Avg. Angular Vel. (rad/s)	–	0.03	–	-0.04	–	-0.03	–	0.14	–	0.12
	Stretch Factor (SF)	1.08x	1.13x	1.76x	1.31x	1.17x	1.21x	1.40x	1.22x	1.27x	1.10x

Table 1. Performance of BiKE-Legs (BL) and DRiVE-Legs (DL) across various obstacle size/fraction and DWA planning horizons.

obstacle radius alone as a proxy for step-over feasibility can therefore be optimistic, allowing the planner to attempt steps that the full-order robot cannot physically clear. It also ignores gait stability considerations, limiting the execution of aggressive stepping motions. The DWA cost weights and thresholds were manually selected; a systematic sensitivity analysis is needed to establish how robust the planner's performance and stepping behavior are to these choices. Future work will incorporate reachability constraints, stability constraints, and foot geometry, along with more efficient implementations to enable longer real-time planning horizons, leading to hardware validation.

DECLARATION OF GENERATIVE AI

ChatGPT was used to generate equations and improve clarity. All content was reviewed by the authors, who take full responsibility for the publication.

REFERENCES

Brock, O. and Khatib, O. (1999). High-speed navigation using the global dynamic window approach. In *Proceedings 1999 ieee international conference on robotics and automation (Cat. No. 99CH36288C)*, volume 1, 341–346. IEEE.

Castillo, G.A., Weng, B., Zhang, W., and Hereid, A. (2021). Robust feedback motion policy design using reinforcement learning on a 3d digit bipedal robot. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5136–5143. IEEE.

Fox, D., Burgard, W., and Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE robotics & automation magazine*, 4(1), 23–33.

Hauser, K., Bretl, T., Latombe, J.C., Harada, K., and Wilcox, B. (2008). Motion planning for legged robots on varied terrain. *The International Journal of Robotics Research*, 27(11-12), 1325–1349.

Herdt, A., Diedam, H., Wieber, P.B., Dimitrov, D., Mombaur, K., and Diehl, M. (2010). Online walking motion generation with automatic footstep placement. *Advanced Robotics*, 24(5-6), 719–737.

Hinojosa, E.H. and Bhounsule, P.A. (2023). Data-driven identification of a non-homogeneous inverted pendulum model for enhanced humanoid control. In *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, 1–8. IEEE.

Huang, W., Kim, J., and Atkeson, C.G. (2013). Energy-based optimal step planning for humanoids. In *2013 IEEE International Conference on Robotics and Automation*, 3124–3129. IEEE.

Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., and Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), eaau5872.

Jelavic, E., Qu, K., Farshidian, F., and Hutter, M. (2023). Lstp: Long short-term motion planning for legged and legged-wheeled systems. *IEEE Transactions on Robotics*, 39(6), 4190–4210.

Kuindersma, S., Deits, R., Fallon, M., Valenzuela, A., Dai, H., Permenter, F., Koolen, T., Marion, P., and Tedrake, R. (2016). Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot. *Autonomous robots*, 40(3), 429–455.

LaValle, S.M. (2006). *Planning Algorithms*. Cambridge University Press. URL `https://msl.cs.uiuc.edu/planning/`.

Norby, J. and Johnson, A.M. (2020). Fast global motion planning for dynamic legged robots. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3829–3836. IEEE.

Ogren, P. and Leonard, N.E. (2005). A convergent dynamic window approach to obstacle avoidance. *IEEE Transactions on Robotics*, 21(2), 188–195.

Peng, C., Paredes, V., and Hereid, A. (2024). Unified path and gait planning for safe bipedal robot navigation. *arXiv preprint arXiv:2403.17347*.

Peng, X.B., Berseth, G., Yin, K., and Van De Panne, M. (2017). Deeploco: Dynamic locomotion skills using hierarchical deep reinforcement learning. *Acm transactions on graphics (tog)*, 36(4), 1–13.

Teng, S., Gong, Y., Grizzle, J.W., and Ghaffari, M. (2021). Toward safety-aware informative motion planning for legged robots. *arXiv preprint arXiv:2103.14252*.

Todorov, E., Erez, T., and Tassa, Y. (2012). Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5026–5033. IEEE.

Zamani, A. and Bhounsule, P.A. (2020). Nonlinear model predictive control of hopping model using approximate step-to-step models for navigation on complex terrain. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3627–3632. IEEE.

Zhang, D., Song, Z., Zhang, F., and Cui, L. (2024). Improving the dynamic window approach for path planning in multi-legged robots. In *2024 2nd International Conference on Artificial Intelligence and Automation Control (AIAC)*. IEEE.